LOGIEDGE: INTELLIGENT EDGE AI PLATFORM FOR COLD-CHAIN LOGISTICS

AIML ZG535 – Machine Learning on Edge

Mini Project Report

Group 27

Team Members

Name	Student ID
KUSH GUNENDRASING MUNOT	2025AE05915
GANESHA V	2025AE05280
ARAVINDHAN G	2025AE05197
ANTO CHRISTAN	2025AE05193

GitHub Repository:
https://github.com/2025ae05280-bot/logibridge.git

Demo Video:
https://drive.google.com/file/d/1nP1H1caHKbgOSAPTIDSSTXHyiDVpxXzs/view?usp=sharing

Deployment Context and Hardware Selection

Summary

LogiEdge must classify cargo on the truck itself: cellular dead zones of 35–90 minutes make any cloud-only design blind exactly when a refrigeration unit fails. This section covers components A (deployment context and architecture) and B (hardware selection) of the AIML ZG535 mini-project, presented by Group 27.

Three conclusions carry the rest of the system:

1. **Edge, not cloud.** A fault raises cargo temperature by about 1 °C per minute and must be alerted within 90 seconds. A cloud round trip fits that budget only while the truck is connected; raw streaming would also cost ₹16.1 lakh a year for the 85-truck pilot.
2. **Raspberry Pi 5 + AI HAT+ (Hailo-8L).** Power is the dominant vertex of the Constraint Triangle. At 7.5 W the Pi is the only option that fits the 10 W AI budget and runs the full Linux service stack, at ₹12.75 lakh for the pilot and ₹39.75 lakh for 265 trucks.
3. **Compute-bound workload.** With the brief's figures, arithmetic intensity is 2.50 FLOP/byte against a ridge point of 1.33, so latency improves by reducing or accelerating arithmetic (pruning, INT8, NPU), not by memory optimisation.

Source material: the problem statement (LogiEdge_MiniProject_Problem_Statement).

1.1 Problem Context

FreightBridge Logistics (Pune) runs 265 trucks across Maharashtra, Karnataka and Andhra Pradesh; the pilot covers the 85 refrigerated pharmaceutical trucks. Today drivers fill temperature log books every 2 hours, and the GPS tracker loses connectivity on rural and hill routes.

Three incidents in 18 months set the requirements:

Incident	What failed	Requirement it creates
₹28 lakh vaccine spoilage, Nashik–Aurangabad	Refrigeration failure went undetected	Detect on the truck within 90 s, even offline
Shipment rejected at a hospital	Incomplete temperature chain-of-custody records	Durable on-device log of every reading window and alert
Two mid-route breakdowns	Engine anomalies visible in sensor data 3 days earlier	Continuous vibration monitoring, not periodic manual checks

The model classifies each 30-second window into one of three classes:

Class	Label	Condition (setpoint 4 °C)	Action
0	Normal	Within ±1 °C, no vibration anomaly	None

Class	Label	Condition (setpoint 4 °C)	Action
1	Warning	1–3 °C off setpoint, or early refrigeration anomaly	Intervene within 2 hours
2	Critical	More than 3 °C off setpoint, or refrigeration failure signature	Escalate immediately

The brief rules out a cloud-dependent architecture: the system must run fully offline and synchronise alert logs when coverage returns.

1.2 Latency

The edge pipeline confirms a fault in about 20 seconds, well inside the 90-second SLA; a cloud path can match that only while the truck has coverage.

Why 90 seconds matters. A failed refrigeration unit raises cargo temperature by about 1 °C per minute. Starting at the 4 °C setpoint, cargo passes the 8 °C upper limit for vaccines (2–8 °C band) in about 4 minutes. The alert must therefore reach the driver and operations centre within 90 seconds of the fault signature appearing in the sensor data.

Edge detection budget (implemented design, data_pipeline/preprocessing.py and inference/decision.py):

Step	Worst case	Note
Wait for the next window to be emitted	≤ 10 s	30 s window, emitted every 10 s
Debounce confirmation	+10 s	Escalation needs 2 of the last 3 windows
Feature extraction + TFLite inference	< 1 ms	Six features, 6→32→16→3 MLP on the CPU
Local MQTT publish + SQLite write	< 0.1 s	Loopback broker, local disk
Total to a confirmed alert	**≈ 20 s**	Leaves about 70 s of the 90 s SLA

A breach of more than 3 °C off the setpoint bypasses the debounce through a hard safety interlock, so a large excursion is escalated on the first window.

Is cloud inference feasible? Only while connected. A rural 4G round trip is typically around 150 ms and can rise to several seconds under packet loss and TCP retransmission (approximate figures, not measured in this project). Even a 15-second worst case would fit inside 90 seconds. The cloud path fails for a different reason: during a dead zone there is no round trip at all (Section 1.4), so the detection time becomes the length of the gap.

Verdict: latency alone does not rule out the cloud; connectivity does. Keeping preprocessing, inference and alerting on the truck makes detection time independent of the network.

1.3 Bandwidth and Cost

Streaming raw telemetry would cost ₹16.1 lakh a year for the pilot; sending only alerts costs under ₹100 a year for the whole 265-truck fleet.

Raw data volume per truck (4-byte float32 samples, payload only):

> $\text{Temperature} = 1\ \text{Hz} \times 4\ \text{B} = 4\ \text{B/s}$
>
> $\text{Vibration} = 500\ \text{Hz} \times 3\ \text{axes} \times 4\ \text{B} = 6{,}000\ \text{B/s}$
>
> $6{,}004\ \text{B/s} \times 86{,}400\ \text{s/day} = 518{,}745{,}600\ \text{B} \approx 518.75\ \text{MB/day}$

Door events are rare and add a negligible amount. JSON keys, timestamps and MQTT headers would make the real figure larger, so this is a lower bound.

Cost at ₹0.10 per MB (M2M SIM tariff from the brief):

Fleet	Raw data per day	Raw cost per day	Raw cost per year	Alerts only, per year
1 truck	518.75 MB	₹51.87	₹18,934	₹0.37
85 trucks (pilot)	44.09 GB	₹4,409	₹16.09 lakh	₹31
265 trucks (full fleet)	137.47 GB	₹13,747	₹50.18 lakh	₹97

The alerts-only column assumes about 10 alert records of up to 1 KB per truck per day. The implemented alert record is smaller still: about 124 bytes of JSON (id, timestamp, truck, class, label, source, probabilities), measured from the payload format in inference/sync_worker.py. Per-window inference messages (about 373 bytes every 10 s) stay on the truck's local broker and are never uplinked.

Reduction: 2.65 MB/day against 137,468 MB/day for the full fleet, a cut of more than 99.99%. The simulator publishes vibration as a 0.5 Hz RMS value, but the physical sensor in the brief samples at 500 Hz, which is the rate a cloud design would have to upload.

1.4 Connectivity and Offline Resilience

A fault that starts inside a dead zone is caught by LogiEdge in about 20 seconds; a cloud-only system would learn about it 35–90 minutes later, after the cargo is lost.

The Nashik–Aurangabad route has seven documented locations where cellular signal disappears for 35–90 minutes. During such a gap a cloud-only design receives no data, runs no inference and sends no alert. Any fault that begins inside the gap is discovered only when the truck reconnects; by then the cargo has passed the 8 °C limit many minutes earlier. This is the pattern behind the ₹28 lakh vaccine loss.

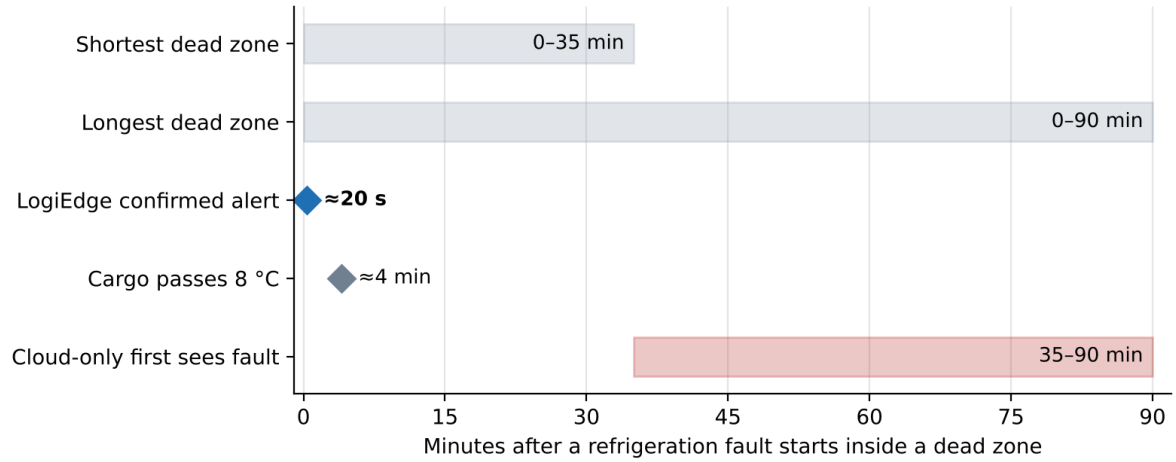


Figure 1.1 — The edge alert lands before the cargo leaves its safe band; a cloud-only system sees nothing until the dead zone ends (dead zones from the brief; edge time from the implemented design).

How LogiEdge behaves in a gap (implemented in inference/alert_store.py and inference/sync_worker.py):

1. Sensors keep publishing to the broker on the truck, so ingestion never depends on the cellular link.
2. The inference service keeps extracting features and classifying every 10 seconds.
3. Every window, alert and door event is written to a local SQLite database (WAL mode) with a synced = 0 flag. This is the chain-of-custody record.
4. A sync worker tries the operations-centre broker every 15 seconds. While the link is down, each attempt fails quietly and the backlog grows on disk.
5. When coverage returns, the worker publishes the unsynced alerts and door events in order, with MQTT QoS 1.
6. A record is marked synced = 1 only after the broker acknowledges it, so an interrupted upload resumes where it stopped and nothing is lost. Event ids let the operations centre remove duplicates.

The broker also holds a retained status message with a last-will "offline" payload, so the operations centre can see which trucks are currently out of contact.

1.5 Privacy and Data Custody

On-device inference keeps raw cargo data inside the truck and sends only small alert records upstream, which shrinks what can be intercepted and gives clients an auditable custody log.

Pharmaceutical clients need proof that cargo condition data cannot be accessed by unauthorised third parties. Edge inference supports this contractually in three ways:

- **Data minimisation.** Raw temperature and vibration streams never leave the truck. Only alert records and door events (about 124 bytes each) cross the cellular network.
- **Custody record.** The SQLite log holds every 30-second feature window, its class and every alert with its timestamp. A rejected-shipment dispute can be answered from this record.

- **Smaller attack surface.** With no continuous raw stream on public cellular infrastructure, there is far less to intercept or scrape.

The lab build is honest about what it does not yet do:

Control	Lab build (this repository)	Production deployment
On-truck broker access	Anonymous connections, bound to localhost (127.0.0.1)	Password file and per-client ACLs
Uplink transport	Plain MQTT on port 1883	MQTT over TLS (port 8883) with client certificates
Alert log at rest	Plain SQLite file	Encrypted filesystem or SQLCipher
Container user	Non-root edge user (UID 1000)	Same, plus read-only model and stats mounts

The report states these boundaries rather than claiming cryptographic protection the lab does not provide.

1.6 System Architecture

Every safety decision is made on the truck; the cellular uplink only carries finished alerts, and only when it is available.

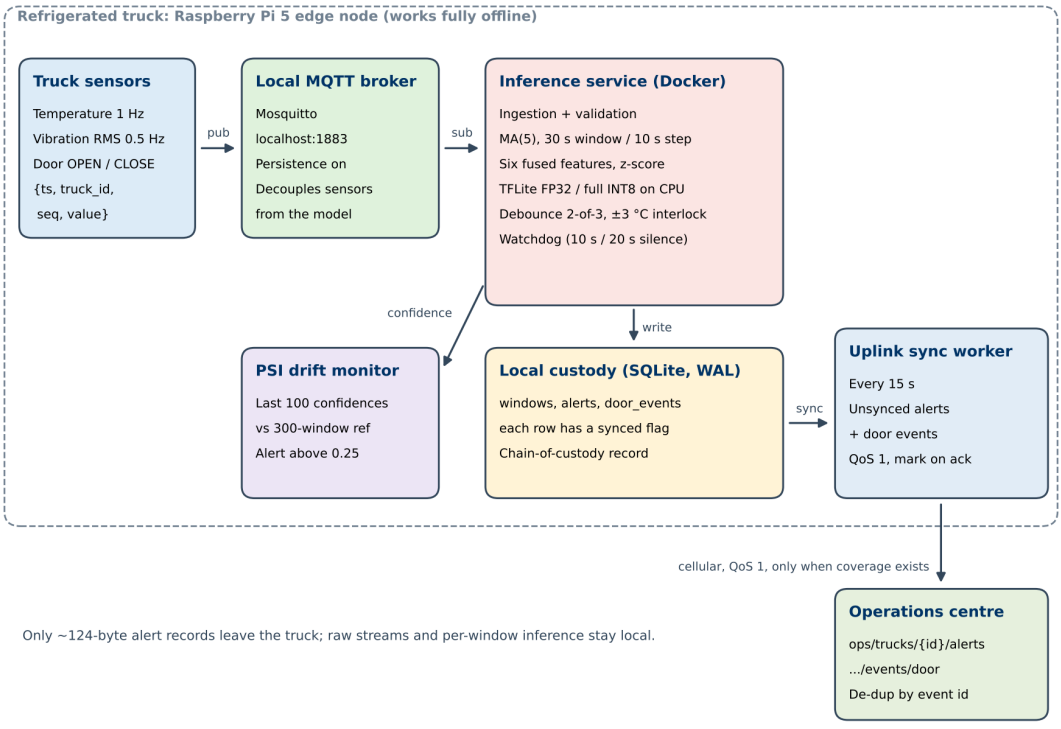


Figure 1.2 — LogiEdge system architecture as implemented: sensors, local broker, inference service, local custody and PSI monitor on the truck; the sync worker forwards alerts to the operations centre when coverage exists.

Component walkthrough, in data-flow order:

Component	What it does	Repository file

Component	What it does	Repository file
Truck sensors (simulated)	Publish temperature (1 Hz), vibration RMS (0.5 Hz) and door OPEN/CLOSE events as {ts, truck_id, seq, value}	data_pipeline/simulator.py
Local MQTT broker	Mosquitto on localhost:1883 with persistence on; decouples sensors from the model	docker-compose.yml, config/mosquitto.conf
Ingestion	Subscribes to logibridge/trucks/{id}/sensors/#, validates payloads, counts sequence gaps, hands messages to a worker thread	inference/inference_service.py
Preprocessing	5-sample moving average; 30 s window emitted every 10 s; six fused features; z-score with stored training_stats.npy	data_pipeline/preprocessing.py
Model runner	TFLite interpreter on the CPU; handles FP32 and full-INT8 models	inference/engine.py, common/tflite_shim.py
Decision logic	Debounce (escalate on 2 of 3 windows, de-escalate after 3); ±3 °C safety interlock	inference/decision.py
Local custody	SQLite (WAL) tables for windows, alerts and door events, each with a synced flag	inference/alert_store.py
Watchdog	Raises a sensor-fault alert if temperature is silent for 10 s or vibration for 20 s	inference/inference_service.py
PSI drift monitor	Compares the rolling distribution of the last 100 confidence scores with a 300-window reference; alert above 0.25	monitoring/drift_monitor.py
Uplink sync worker	Every 15 s, forwards unsynced alerts and door events to the operations broker with QoS 1	inference/sync_worker.py
Operations centre	Receives ops/trucks/{id}/alerts and ops/trucks/{id}/events/door; de-duplicates by id	config/ops_broker.conf (lab stand-in)

Topics on the truck: logibridge/trucks/{id}/sensors/{temperature,vibration,door} in; …/inference (every window), …/alerts (on escalation), …/status (retained, last-will) and …/drift (PSI alerts) out.

1.7 Constraint Triangle and Hardware Selection

Power is the dominant vertex, and only the Raspberry Pi 5 with the AI HAT+ satisfies it while also running the full service stack at a viable fleet cost.

The three vertices for this deployment:

- **Power (dominant).** The AI node draws from the truck's 12 V supply through a DC-DC converter, with a hard 10 W AI budget, inside an unventilated cabin. A hard limit removes an option outright, whatever its other merits.
- **Cost (second filter).** Every rupee per unit is multiplied by 85 trucks in the pilot and 265 at full scale, so cost ranks the options that pass the power limit.
- **Performance (easily met).** Even the brief's 45 MFLOP workload needs about 3 ms on the Pi CPU, against a 90-second SLA. Our trained model is far smaller still.

Option	AI compute	Power vs 10 W budget	Unit price	85 trucks (pilot)	265 trucks (fleet)	Runs the service stack	Verdict
Raspberry Pi 5 (8 GB) + AI HAT+ (Hailo-8L)	13 TOPS NPU + quad-core CPU	7.5 W, within	~₹15,000	₹12.75 lakh	₹39.75 lakh	Yes: Linux, Docker, Mosquitto, SQLite	**Selected**
Jetson Orin Nano Super Developer Kit	67 TOPS	15 W at moderate load, over	~₹45,000	₹38.25 lakh	₹1.19 crore	Yes	**Rejected**
STM32H7-based custom MCU	Cortex-M7, ~1 MB SRAM	0.4 W, within	~₹3,500	₹2.98 lakh	₹9.28 lakh	No OS, no Docker	**Rejected**

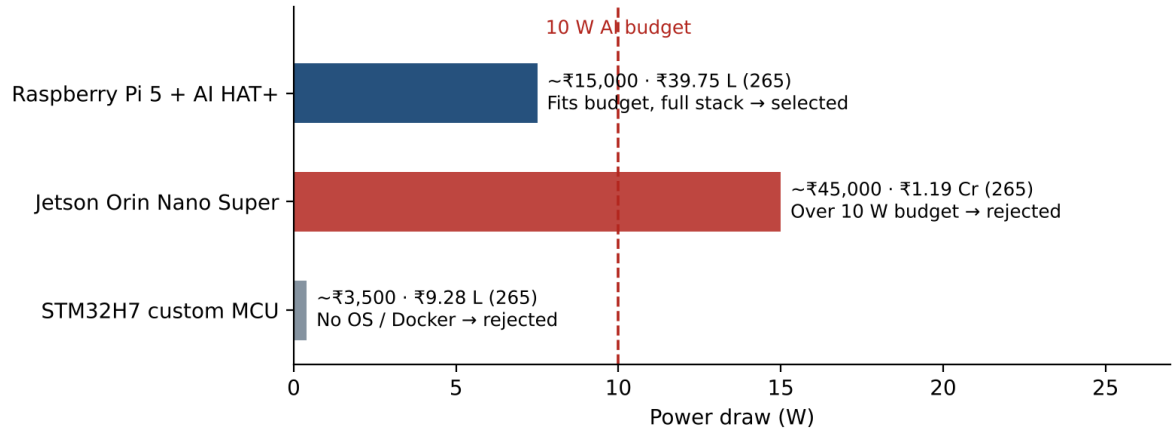


Figure 1.3 — Power draw of the three candidates against the 10 W AI budget, with unit price and 265-truck fleet cost.

For the Raspberry Pi 5 + AI HAT+. Its 7.5 W fits the 10 W budget with margin for the DC-DC converter's losses. Linux hosts the whole stack that the edge design depends on: the local Mosquitto broker, the Docker inference container, the SQLite custody log, the PSI monitor and OTA updates through Ansible. The TFLite model runs in under a millisecond on the CPU, so the 90 s SLA is dominated by windowing (Section 1.2), not by inference. The Hailo-8L NPU is not used by the current model; it is headroom for larger models later.

Against the Jetson Orin Nano Super. At 15 W under moderate load it breaks the power budget and would need active cooling in a hot cabin. Its 7 W mode would have to be benchmarked on this workload before it could be considered. At ~₹45,000 per unit it costs three times the Pi: ₹1.19 crore for the full fleet, for 67 TOPS that a six-feature MLP does not need.

Against the STM32H7 custom MCU. It is the cheapest and lowest-power option, and our small MLP itself would fit. But a Cortex-M7 without an MMU or operating system cannot run Docker, an MQTT broker, SQLite or the Ansible-driven OTA process, so the system would have to be rewritten as bare-metal firmware. Its ~1 MB SRAM is also far below the 18 MB per-inference working set the brief assumes. (A 30-second buffer of raw 500 Hz three-axis vibration is only about 176 KB, so buffering alone is not the reason to reject it.)

Decision: Raspberry Pi 5 (8 GB) + AI HAT+ for the 85-truck pilot, with the same unit for the 265-truck rollout.

1.8 Arithmetic Intensity and Roofline

With the brief's figures the workload is compute-bound (2.50 FLOP/byte against a ridge point of 1.33), so latency improves by reducing or accelerating arithmetic.

Inputs from the brief: about 45 MFLOP and 18 MB of data (weights + activations) per inference; Raspberry Pi 5 CPU peak about 16 GFLOP/s (NEON SIMD) and memory bandwidth about 12 GB/s (LPDDR4X).

$$I = W / Q = (45 \times 10^6 \text{ FLOP}) / (18 \times 10^6 \text{ B}) = 2.50 \text{ FLOP/B}$$

$$I_{ridge} = P_{max} / B_{max} = (16 \times 10^9 \text{ FLOP/s}) / (12 \times 10^9 \text{ B/s}) = 1.33 \text{ FLOP/B}$$

$$P_{attainable} = \min(P_{max}, I \times B_{max}) = \min(16, 30) = 16 \text{ GFLOP/s} \Rightarrow t \approx 45 \times 10^6 / 16 \times 10^9 \approx 2.8 \text{ ms}$$

Classification. I = 2.50 is to the right of the ridge point, on the flat part of the roof: the CPU's arithmetic throughput limits speed, not memory bandwidth. Moving data faster would not help; doing less arithmetic, or doing it faster, would.

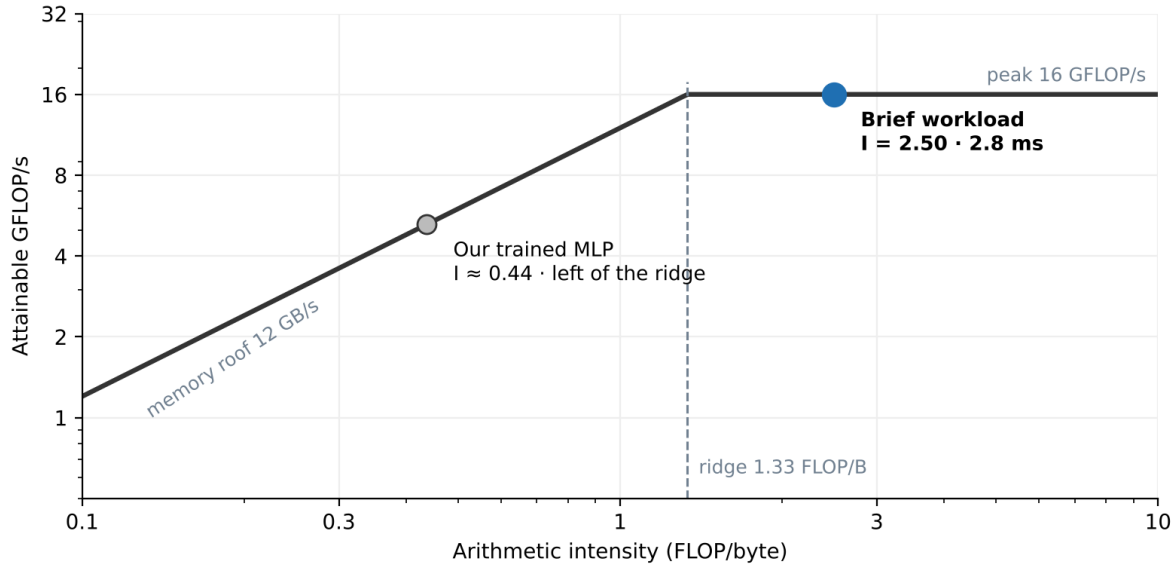


Figure 1.4 — Raspberry Pi 5 CPU roofline (log–log). The brief's workload sits on the flat compute roof; our much smaller MLP sits on the memory slope, but at about 1.5 kFLOP its absolute cost is negligible.

What the Roofline result says to optimise:

Optimisation	Effect on a compute-bound workload	Where it appears in the project
Structured pruning	Removes whole units, so fewer FLOPs per inference	M3: 35% of hidden units removed (PolynomialDecay schedule)
INT8 quantisation	A 128-bit NEON register holds 16 int8 values instead of 4 float32, so up to 4× more operations per instruction; also cuts memory traffic 4×	M2 and M3: full-INT8 post-training quantisation
NPU offload	Moves the arithmetic to the 13 TOPS Hailo-	Not implemented; headroom for

Optimisation	Effect on a compute-bound workload	Where it appears in the project
	8L	larger models

Because the workload is compute-bound, INT8's main benefit here is faster arithmetic; its 4× smaller memory traffic matters less.

Our trained model is a different case. The deployed 6→32→16→3 MLP has 803 parameters and about 752 multiply-adds (about 1.5 kFLOP) per inference. Its weights are about 3.2 KB in float32, so its arithmetic intensity is roughly 0.45 FLOP/byte, to the left of the ridge. In absolute terms it is tiny: compute and memory time are both well under a microsecond, so measured latency is dominated by interpreter overhead. The team's laptop benchmark (optimisation/results/benchmark_results.csv, ai-edge-litert 1.2.0 runtime) records mean latencies of 0.0016 ms (M1 FP32), 0.0060 ms (M2 INT8) and 0.0140 ms (M3 pruned INT8); see Section 4. These are laptop figures; latency still has to be measured on the Pi itself.

Sensor Pipeline and MQTT Design

Three simulated streams publish to a broker on the truck, a fixed preprocessing chain turns every 30-second window into six features, and one model sees all six at once (component C, Module 3).

2.1 Sensor Simulator

`data_pipeline/simulator.py` generates the three streams from the brief and publishes them to Mosquitto on localhost:1883 as JSON `{ts, truck_id, seq, value}`. The `seq` counter lets the inference service detect dropped messages.

Stream	Rate	Normal	Anomaly (--anomaly)
temperature	1 Hz	N(4.0 °C, 0.3), setpoint 4 °C	temp_drift: +0.08 °C per reading
vibration_rms	0.5 Hz	N(0.45 g, 0.05), compressor	vibration: step to N(1.2 g, 0.15), bearing wear
door	On change	OPEN every 5–15 min, CLOSE after 30–120 s	combined: both anomalies at once

`--anomaly {none|temp_drift|vibration|combined}` selects the mode. To keep the data realistic, an open door warms the cargo by 0.5–1.5 °C and 0.5% of temperature readings carry a ±1 °C spike; `--seed` makes runs reproducible.

2.2 MQTT Topic Tree and QoS

Every topic sits under one truck id, so one wildcard selects a truck (`logibridge/trucks/T01/#`) or a fleet-wide signal (`logibridge/trucks/+/alerts`).

On the truck: Mosquitto, localhost:1883

	QoS	publisher → subscriber
logibridge/trucks/{truck_id}/		
├─ sensors/		
│ ├─ temperature	**QoS 1**	simulator → inference (1 Hz)
│ ├─ vibration	**QoS 1**	simulator → inference (0.5 Hz)
│ └─ door	**QoS 1**	simulator → inference → SQLite
├─ inference	**QoS 1**	inference → drift monitor (every 10 s)
├─ alerts	**QoS 2**	inference → driver display
├─ status	**QoS 1 · retained · LWT**	inference → any observer
└─ drift	**QoS 1**	drift monitor → observers (PSI > 0.25)

Cellular uplink, only when coverage exists (SQLite backlog, synced flag)

ops/trucks/{truck_id}/		
├─ alerts	**QoS 1**	sync worker → ops centre
└─ events/door	**QoS 1**	sync worker → ops centre

Figure 2.1 — MQTT topic tree as implemented, with the QoS, publisher and subscriber of each topic. Only the two ops/ topics cross the cellular network.

Topic	QoS	Why this level
sensors/temperature, sensors/vibration	1	A lost sample can leave a window below its 25-sample (temperature) or 12-sample (vibration) minimum and delay the decision by 10 s. On the loopback link the acknowledgement costs nothing.
sensors/door	1	Rare, and every event is part of the custody record.
inference	1	PSI needs an unbroken run of the last 100 confidence scores.
alerts (truck)	2	Exactly once: a duplicated Critical alert pages the driver twice. Alerts are rare, so the four-step handshake costs nothing.
status (retained, LWT)	1	A new subscriber sees at once whether the truck is up; the broker publishes "offline" itself if the service dies.
drift	1	Infrequent maintenance signal that must arrive.
ops/…/alerts, ops/…/events/door	1	Uplink over cellular; see below.

The uplink is QoS 1, not 2, on purpose. QoS 2's four-message handshake keeps breaking on a lossy rural link. Instead the sync worker marks a record synced only after the broker acknowledges it, and the operations centre drops repeated ids: exactly-once in effect, without the handshake. Durability through a dead zone comes from SQLite, not QoS, because MQTT sessions here do not persist across reconnects.

2.3 Preprocessing Design

`data_pipeline/preprocessing.py` serves both dataset generation and the live service, so features cannot diverge.

- **5-sample moving average.** Removes spikes and noise; its ~2 s lag is negligible against the 90 s SLA.
- **30 s window, 10 s step.** 30 temperature and 15 vibration samples give stable statistics; a decision every 10 s plus the 2-of-3 debounce keeps a confirmed alert at about 20 s (Section 1.2).
- **Six features, each tied to a fault.** Temperature mean (breach), standard deviation (door openings, unstable control), rate of change in °C/min from a least-squares slope (drift before it becomes a breach), vibration RMS (compressor load), peak (shocks) and kurtosis (impulsive bearing faults; the simulator's bearing-wear step shows mainly in RMS and peak).
- **Frozen z-score.** Mean and standard deviation come from the first 10 minutes of a clean Normal run, saved once as `training_stats.npy` and loaded at start-up; Ansible ships the file with the model. Recomputing from live data would let a slow drift normalise itself away: the statistics would follow the fault and the model would keep seeing "normal".

2.4 Normalisation Experiment

`data_pipeline/normalisation_experiment.py` ran the M1 FP32 model on the 453 validation windows (117 Normal, 168 Warning, 168 Critical) with the correct statistics, with the stored mean shifted by 3σ in each direction, and with the standard deviation tripled (`results/normalisation_experiment.csv`).

Statistics used	Accuracy	Normal recall	Warning recall	Critical recall
Correct	100.0%	100.0%	100.0%	100.0%
Mean + 3σ	99.8%	100.0%	100.0%	99.4%
Mean − 3σ	82.3%	34.2%	98.8%	99.4%
Std × 3	98.2%	100.0%	97.6%	97.6%

The damage is one-sided, and that follows from where the classes sit. Lowering the stored mean by 3σ adds +3 to every z-score, so a Normal window looks like a truck already about 0.9 °C warm with 0.04 g more vibration: 77 of 117 Normal windows become false Warnings. That is alarm fatigue. Raising the mean barely matters, because anomalous windows sit tens of σ above the clean mean. Tripling the standard deviation pulls every z-score towards zero and misses 4 Critical windows, taking Critical recall to 97.6%, close to the 95% floor in Task F3. The statistics are part of the model and ship with it.

2.5 Data Fusion Choice

Temperature and vibration features are computed separately and then concatenated into one 6-value vector (feature-level fusion).

Level	What it would mean here	Verdict
Data-level	Merge raw samples into one input before extracting features	Rejected: the 1 Hz and 0.5 Hz streams (500 Hz on the real sensor) need resampling to one clock, and the input grows from 6 to 45 values per window, or about 45,000 at 500 Hz
Decision-level	One model per sensor, then combine their votes	Rejected: the combined (Critical) signature needs both signals together; mild drift plus mild vibration can be Critical while neither model alone fires
Feature-level	Six features from two sensors, one model	Chosen

Each sensor keeps its own rate and filter, and the model gets a small, fixed, physically meaningful input: the 6→32→16→3 MLP runs in under a millisecond at 100% validation accuracy, and the logged features show an auditor why a window was flagged. Door events stay out of the vector; they are rare and already explicit in the SQLite log.

Model Deployment and Pipeline Mapping

A model update reaches a truck as a 5.63 kB image layer, not a 237 MB image, and every stage of the Edge ML pipeline runs on the truck (components D and E3, Modules 4 and 5).

3.1 Dataset and Training

`training/generate_dataset.py` ran the simulator five times per mode and labelled each 30 s window by mode: none → Normal, temp_drift and vibration → Warning, combined and cooling_fault → Critical. The split is by run (3 train, 1 validation, 1 test per mode), so overlapping windows from one run can never sit on both sides.

Item	Value	Source
Windows	2,265 from 25 runs (train 1,359 · val 453 · test 453)	training/data/dataset.csv
Model	MLP 6 → 32 → 16 → 3, ReLU, softmax	training/train_model.py
Validation accuracy	100% (target > 88%)	training/results/metrics.json
Test accuracy (unseen runs)	99.3%; Critical recall 98.8%	training/results/metrics.json

100% reflects cleanly separable simulated classes, not a field-ready model. The three test errors are all fault windows called Normal (one Warning, two Critical), the unsafe direction, which is why the decision layer keeps the ±3 °C interlock regardless of the model.

3.2 Docker Layer Cache and OTA Saving

`inference/Dockerfile` starts from python:3.11-slim, installs the pip requirements first, copies the code, and copies `model.tflite` as the very last instruction. `MODEL_PATH` selects the variant at run time, and results are published to `logibridge/trucks/{truck_id}/inference`. In the demo the same image switched from M1 to M2 by changing only `MODEL_PATH`, with no rebuild, and the model name in each inference message changed with it.

Layer (top = last)	Size	After swapping M1 → M3
COPY model.tflite	5.63 kB	Rebuilt (new layer id)
Application code (3 COPY layers)	≈ 39 kB	Using cache
pip install requirements	81.6 MB	Using cache
python:3.11-slim base	≈ 155 MB	Using cache
Whole image	236.6 MB	Every layer below the model has the same id

Saving for 85 trucks. Changing only the model rebuilt only the top layer; `podman history` shows the same parent layer id under both images. Sending that layer to 85 trucks is 85 × 5.63 kB = 0.48 MB, about ₹0.05 per update, against 20.1 GB and ₹2,011 for the full image: a saving of 99.998%.

3.3 The 10-Stage Edge ML Pipeline

Stage	What happens in LogiEdge
1 Problem framing	Classify each 30 s window as Normal, Warning or Critical on the truck, alerting within the 90 s SLA with no network dependency.
2 Data collection	The simulator publishes temperature, vibration and door events to the truck's Mosquitto broker; dataset runs replay it in each mode.
3 Preprocessing	A 5-sample moving average removes spikes and noise; malformed or non-finite payloads are dropped and counted.
4 Feature engineering	Six fused features per 30 s window, z-scored with the frozen training_stats.npy.
5 Model design	A 6→32→16→3 MLP: 803 parameters, small enough for any candidate device.
6 Training and validation	Run-level split; passed the 88% accuracy gate and the 95% Critical-recall gate.
7 Optimisation	M2 full-INT8 PTQ with 250 calibration windows; M3 35% structured pruning then INT8 (Section 4).
8 Conversion and packaging	TFLite models in a python:3.11-slim image, model copied last so an update is one small layer.
9 Deployment	Ansible pulls the image from the local registry and starts it with MODEL_PATH, STATS_PATH and TRUCK_ID.
10 Monitoring	PSI on confidence scores, a sensor watchdog, and the SQLite custody log synced to the operations centre.

3.4 OTA Strategy

Planning figures from the brief: a 280 KB INT8 model every 6 weeks to 85 trucks at ₹0.10/MB, so one complete transfer is 85 × 0.28 MB = 23.8 MB = ₹2.38.

Strategy	Cost per update cycle	Verdict
Full replacement	₹2.38 (23.8 MB)	Rejected: an unverified model reaches all 85 trucks at once
Canary (10 trucks first)	₹0.28 + ₹2.10 = ₹2.38	Selected: a bad model can reach only 10 trucks
Shadow mode	₹2.38 + comparison logs	Rejected: the candidate cannot act on alerts, and the logs add uplink traffic

Recommendation: canary. All three cost the same to transfer, so the decision is about risk. For vaccines, a model that misses Critical windows on every truck is the worst outcome, and canary caps it at 10 trucks for one gate period, during which Critical recall and watchdog health are checked. Rural dead zones also mean a fleet-wide rollout would finish unevenly; a canary gate tolerates that, because trucks that have not updated keep running the previous model and logging locally. Shadow mode is useful for evaluation but does not deliver the update.

Optimisation and Pareto Analysis

All three variants pass the safety requirements. M1 provides the lowest latency, while M3 provides the smallest model size with no loss in Critical Recall. Because all variants comfortably satisfy the deployment SLA, M3 is selected for deployment.

4.1 Variants and Benchmark

M1 is the FP32 baseline. M2 is full-INT8 post-training quantisation with 250 class-balanced calibration windows. M3 applies 35% structured pruning with a PolynomialDecay schedule, then the same INT8 conversion. `optimisation/benchmark.py` timed 200 runs after 10 discarded warm-ups on the ai-edge-litert 1.2.0 interpreter, and estimated energy as E = P × t, with P = 45 W laptop TDP × measured CPU share.

Variant	Mean latency (ms)	p95 latency (ms)	Size (KB)	Accuracy (%)	Energy (mJ)
M1 FP32	0.0016	0.0017	5.19	100.0	0.007
M2 PTQ INT8	0.0060	0.0062	3.38	99.6	0.240
M3 Pruned + INT8	0.0140	0.0144	2.94	100.0	0.604

A variant is Pareto-optimal when no other is at least as good on latency, size and accuracy and better on one of them. All three qualify: M1 is fastest, M3 is smallest with full accuracy, and M2 sits between them on size. M2 still loses to M1 on both latency and accuracy, so its only claim is a 35% smaller file.

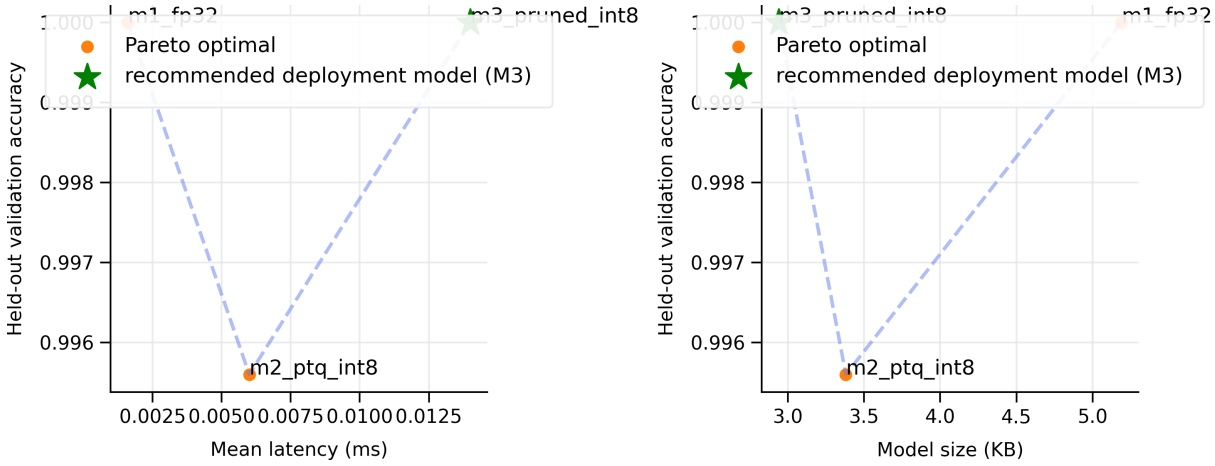


Figure 4.1 — Pareto view of validation accuracy against mean latency (left) and model size (right). All three variants are Pareto-optimal. M1 provides the lowest latency, while M3 provides the smallest model size while maintaining 100% accuracy and Critical Recall. Because all variants comfortably satisfy the SLA, M3 is selected as the deployment recommendation.

Reading the results. INT8 is slower here, which looks like a contradiction of the Roofline advice in Section 1.8 but is not. That advice applies to the brief's 45 MFLOP workload. Our MLP does about 750 multiply-adds, so the quantise and dequantise steps around each INT8 call cost more than the arithmetic they save. Pruning removes units but adds no speed at this size; its benefit is the smallest file. Energy follows latency, and the values are laptop estimates, not Pi measurements.

4.2 Deployment Recommendation

Which model should go to the 85 trucks? M3 (Pruned INT8).

The recommendation is based on the three criteria defined in the brief:

- **Latency budget.** A new window arrives every 10 seconds, while the system must raise an alert within 90 seconds. M3's p95 latency of 0.0144 ms remains several orders of magnitude below both limits, making inference time negligible relative to windowing and debounce delays.
- **Flash and SRAM.** M3 is the smallest model at 2.94 KB, compared with 3.38 KB for M2 and 5.19 KB for M1. While all three comfortably fit on the Raspberry Pi 5, selecting the smallest model minimizes storage requirements and provides greater flexibility for future deployments on more constrained hardware.
- **Critical recall.** M3 achieves 100% Critical Recall on the validation set, exceeding the project requirement of 95%. Its safety performance is identical to M1 while requiring substantially less storage.

Recommendation: M3 (Pruned INT8) should be deployed to the 85-truck fleet because it meets the SLA, achieves 100% Critical Recall, and provides the smallest model footprint. M2 remains the fallback option at 3.38 KB and approximately 99% accuracy if deployment constraints change. These benchmark results were collected on a laptop rather than the Raspberry Pi 5 deployment target. Before pilot deployment, the same evaluation should be repeated on the Pi 5 to validate real-device performance.

MLOps Monitoring and Reflection

5.1 PSI Drift Monitoring

`monitoring/drift_monitor.py` built the reference from 300 clean Normal windows of M1 confidence scores in four bins (97.3% fall in [0.50, 0.75)). It computes PSI over the last 100 inferences every 60 s and prints [LOGIBRIDGE DRIFT ALERT] above 0.25. In the demo the simulator ran ten times faster than real time.

Phase	PSI	Result
Before: clean data	0.015	Stable, below 0.10
During: --anomaly combined, first check	0.322	DRIFT ALERT within one 60 s check (limit 5 min)
During: fault continues	3.07	Peak
After: clean data restored	1.44 → 0.010	Below 0.10 once clean windows refill the buffer

Drift type. This is sudden covariate (data) drift: the input distribution changed in one step while the mapping from inputs to labels did not, so it is not concept drift. Because PSI is computed on output confidence, it detects the input shift indirectly, through the model's response to it.

5.2 Ansible Idempotency

`deployment/logibridge_deploy.yml` has exactly seven tasks: create the runtime directory, copy the model and stats, copy reference_dist.json, stop the container if files changed, pull the image from the local registry, start the container with its environment variables, and wait 15 s and verify it is running. On a fresh truck the first run reported **changed=4**; the second run, with nothing changed in between, reported **changed=0**, and the container stayed healthy.

5.3 Reflection

One genuine technical difficulty. The second Ansible run kept reporting changes even though nothing had changed. We run Podman, and its Docker-compatible API reports the network mode and volume fields differently from Docker, so the docker_container module saw a difference on every run and recreated the container. We excluded only those two fields from the comparison, keeping the image, network and environment checks, and made the stop task conditional on a file change. The second run then reported changed=0.

One architectural change. PSI on output confidence cannot tell a refrigeration fault from model drift: the injected fault raised PSI to 3.07, exactly as a degraded model would. We would compute PSI per input feature, and only on windows the decision layer accepted as Normal. A real fault would then raise an alert without raising a drift flag, a drift flag would point at the sensor that moved, and the six-weekly OTA cycle would receive a retraining signal it could trust.